

Solution Requirements

Date	16 September 2026
Team ID	SWTID-2026-2825
Project Name	AI WeatherWise
Maximum Marks	4 Marks

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	Users register and log in with email & password; passwords are hashed with bcrypt and a JWT is issued on successful login.	High
2	Authorization levels	Role-based access with a default <code>reader</code> role for registered users and an <code>admin</code> role for elevated operations, enforced via authentication middleware.	High
3	External interfaces	Integrates with the OpenWeatherMap REST API for live weather metrics and Google Gemini AI (<code>@google/genai</code>) for natural-language insight generation.	High
4	Transactions / data processing	Full CRUD on favorite Location documents, each linked to its owning User via a foreign-key reference.	High
5	Reporting	Returns structured JSON weather summaries and AI-generated recommendation text for each request, suitable for any client to render.	Medium
6	Business rules	A user may only view or modify their own saved locations; an AI insight request must be preceded by a successful weather fetch so the prompt has live context.	High
7	Compliance to laws or regulations	Passwords are never stored in plaintext (bcrypt hashing); JWT secrets and third-party API keys are kept in <code>.env</code> and excluded from version control via <code>.gitignore</code> .	High
8	Other	A resilient fallback mode returns a safe default response when an external API key (Gemini/OpenWeatherMap) is missing or unavailable, preventing a full service outage.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Auth and Location endpoints (MongoDB-only) should respond quickly; Weather/AI endpoints depend on external API latency.	Avg response < 2 seconds (auth/location); < 5 seconds (weather/AI)
2	Scalability	Stateless JWT authentication allows multiple Node.js instances to run behind a load balancer with no shared session state.	Designed to support 500+ concurrent users
3	Security & Data Privacy	bcrypt password hashing, JWT-based sessionless auth, and request payload sanitization to block NoSQL injection and XSS vectors.	No plaintext password storage; all inputs sanitized
4	Reliability & Availability	Centralized error-handling middleware plus a fallback mode for weather/AI API outages.	Graceful degradation instead of crashes; verified in Performance Testing
5	Usability & Accessibility	Consistent, well-documented JSON REST responses, verified with a shared Thunder Client / Postman collection.	Documented in <code>postman_collection.json</code>
6	Other (Maintainability)	Modular MVC structure (routes / controllers / services / models) so new endpoints can be added without touching existing modules.	New feature addable in a single new controller + route + service